.

بنام آنکه جان را فکرت آموخت



گزارش حل تمرین ساخت یک هایپروایزر میناتوری روی KVM

اسم : احمد شکیب حیدری
شماره دانشجویی ۴۰۲۳۱۴۴۵
درس : رایانش ابری

هدف :

هدف این تمرین آشنایی عملی ما با مجازی‌سازی در لینوکس و نحوه کار KVM بود. در این پروژه یک هایپروایزر ساده به نام minivmm را تکمیل کردم و یاد گرفتم چگونه ماشین مجازی بسازیم، پردازنده‌ها و حافظه آن را مدیریت کنم و خروج‌های ماشین مجازی را بررسی کنیم. همچنین با ابزارهای htop ،strace ،vmctl و QEMU کار کردیم.

مشخصات سیستم من :

```
PRETTY_NAME="Ubuntu 26.04.1 LTS"
CPU:
Architecture:                            x86_64
CPU(s):                                  20
Model name:                              12th Gen Intel(R) Core(TM) i7-12700H
Thread(s) per core:                      2
Core(s) per socket:                      14
NUMA node0 CPU(s):                       0-19
MEMORY:
               total        used        free      shared  buff/cache   available
Mem:            14Gi       7.1Gi       568Mi       166Mi       7.7Gi       7.8Gi
Swap:          4.0Gi        53Mi       3.9Gi
VIRTUALIZATION:
crw-rw----+ 1 root kvm 10, 232 Oct 30 18:28 /dev/kvm
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$
```

گزارش قسمت پیاده سازی minivmm

بخش اول قسمت‌های ناقص فایل minivmm.c را تکمیل شد. باید چهار تابع و پیاده سازی و عملکرد درست آن را هم بررسی کردیم

بعدش با باز کردن /dev/kvm/ ارتباط با KVM برقرار کردیم. سپس ماشین مجازی ساخته ساختیم، حافظه آن با mmap آماده شد و برنامه مهمان در حافظه قرار داده شد.

بعد پردازنده‌های مجازی ساخته را ساختیم و رجیسترهای آن‌ها تنظیم شدند. در نهایت با دستور KVM_RUN اجرای مهمان آغاز شد.

تابع handle_io , vcpu را برای مدیریت عملیات ورودی و خروجی و تابع‌های count_exit و وprint_exit_table را برای شمارش و نمایش خروج‌های ماشین مجازی تکمیل کردیم.

آزمایش یک :

با دستورات

make clean

make

make test

```
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ make clean
rm -f core/minivmm guests/*.bin
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ make
cc -O2 -g -Wall -Wextra -std=gnu11 -o core/minivmm core/minivmm.c -pthread
nasm -f bin -o guests/hello.bin guests/hello.asm
nasm -f bin -o guests/cpu_burn.bin guests/cpu_burn.asm
nasm -f bin -o guests/mem_touch.bin guests/mem_touch.asm
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ make test
./core/minivmm --mem 16 --vcpus 1 guests/hello.bin
[minivmm] pid=29557 guest=guests/hello.bin mem=16MiB vcpus=1
Hello from the guest! I am running with no OS at all.

[minivmm] ===== VM exits seen by userspace (0.10 s) =====
reason                     vcpu0        total
IO                            54           54
HLT                            1            1
TOTAL                         55           55
[minivmm] VM halted
```

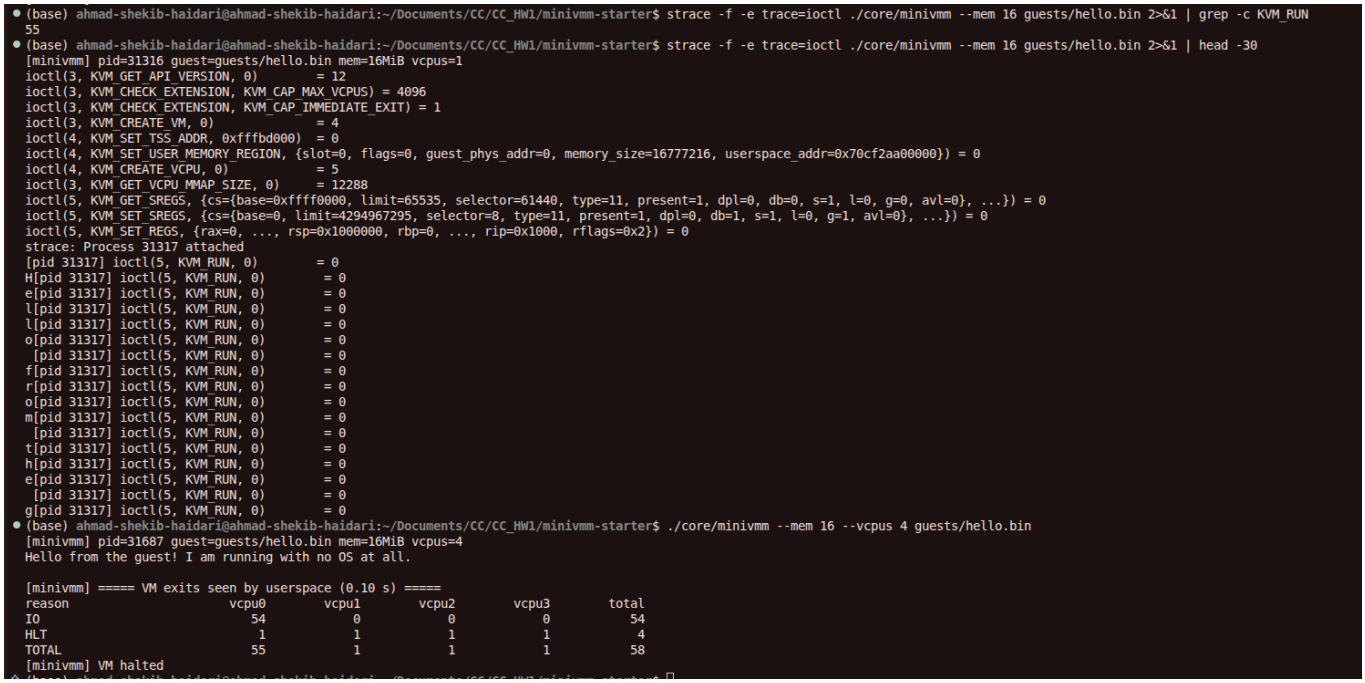
خروج IO مربوط به عملیات ورودی و خروجی مهمان بود و خروج HLT نشان داد که پردازنده مجازی اجرای خود را به پایان رسانده است.

در پایان پیام VM halted نمایش داده شده است. که نشان از کار کرد درستش هست

آزمایش دوم

با دستور

strace -f -e trace=ioctl ./core/minivmm --mem 16 guests/hello.bin 2>&1 | grep -c KVM_RUN



تعداد خطوط مربوط به KVM_RUN برابر ۵۵ بود.

این مقدار با مجموع خروج‌های ثبت‌شده در جدول VM Exit برابر شد.

در اجرای چهار vCPU نیز متن مهمان فقط یک بار چاپ شد و چهار خروج HLT ثبت گردید. مجموع خروج‌ها در این حالت ۵۸ بود.

نتیجه: برنامه می‌تواند عملیات خروج از ماشین مجازی را مدیریت کند و بعد از رسیدگی به I/O، اجرای مهمان را ادامه دهد.

آزمایش سوم :

در این بخش از پنل کنترل vmctl.py برای مدیریت ماشین‌های مجازی استفاده کردیم.

این پنل امکان ساخت، اجرا، مشاهده وضعیت، تغییر منابع و توقف ماشین‌های مجازی را فراهم می‌کند.

دو ماشین با نام‌های cpu1 و cpu4 را اجرا کردیم.

بعدش با دستور

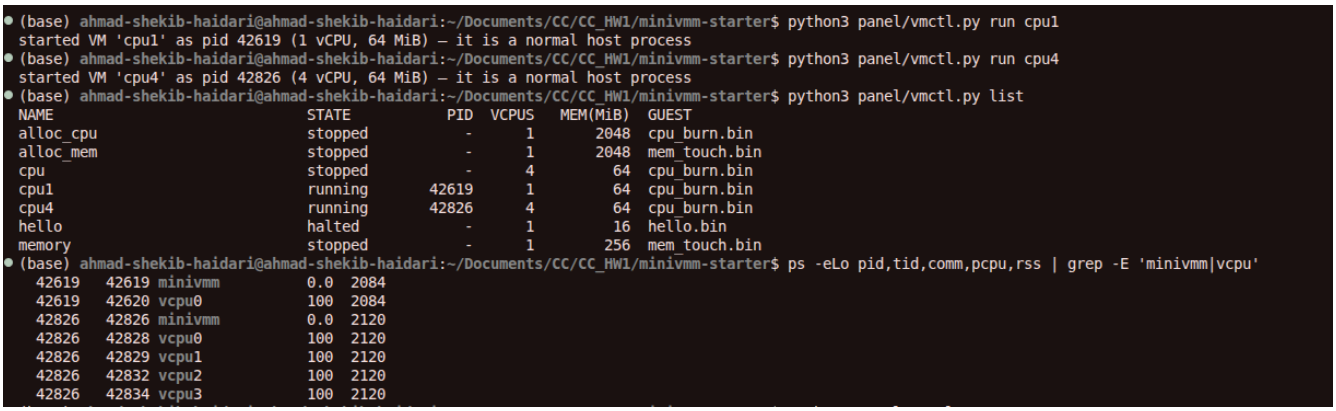
ps -eLo pid,tid,comm,pcpu,rss | grep -E 'minivmm|vcpu

'

نخ ها را بررسی کردیم.

مشاهده کردیم که ماشین cpu1 دارای یک نخ vCPU و ماشین cpu4 دارای چهار نخ vCPU بود.

نتیجه: هر ماشین مجازی به‌صورت یک فرآیند میزبان اجرا می‌شود و پردازنده‌های مجازی آن از طریق نخ‌های جداگانه مدیریت می‌شوند.



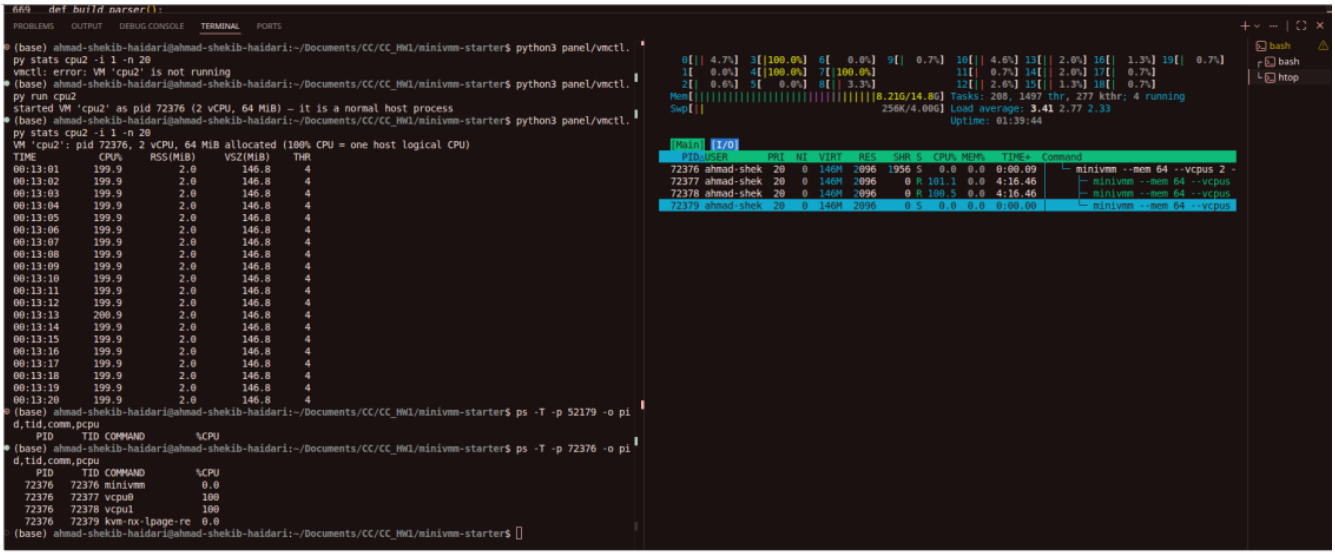
آزمایش چهارم :

در این آزمایش برنامه cpu_burn.bin را با تعداد مختلف پردازنده مجازی اجرا کردیم.

برای بررسی منابع از vmctl stats و htop استفاده کردیم.

در آزمایش دو vCPU، مصرف CPU حدود ۲۰۰ درصد بود و در htop دو نخ فعال مشاهده شد که هرکدام تقریباً ۱۰۰ درصد CPU مصرف می‌کردند.

نتیجه: با افزایش تعداد vCPUها در برنامه cpu_burn، مصرف CPU تقریباً به همان نسبت افزایش پیدا کرد.



آزمایش پتجم :

الف) تغییر تعداد vCPU

ابتدا ماشین cpu1 را با یک پردازنده مجازی اجرا کردیم. سپس با دستور زیر تعداد پردازنده‌ها را به چهار تغییر دادیم:

```
python3 panel/vmctl.py resize cpu1 --vcpus 4 --restart
```

در این آزمایش ماشین ابتدا متوقف و دوباره اجرا شد. بنابراین تغییر منابع از نوع Cold Resize بود.

ب) تغییر حافظه RAM

ماشین m را ابتدا با ۶۴ مگابایت RAM و فایل mem_touch.bin اجرا کردیم.

مقدار RSS برابر ۶۵ مگابایت شد.

سپس دستور زیر را اجرا کردیم:

```
python3 panel/vmctl.py resize m --mem 256 --restart
```

بعد از تغییر حافظه، مقدار RSS به‌تدریج از ۸۱ به ۲۵۷ مگابایت افزایش یافت.

نتیجه: حافظه فیزیکی مصرف‌شده با دسترسی مهمان به صفحات حافظه افزایش پیدا می‌کند و تمام RAM تعریف‌شده الزاماً از همان ابتدا مصرف نمی‌شود.

```
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py stats m -n 8
VM 'm': pid 84501, 1 vCPU, 64 MiB allocated (100% CPU = one host logical CPU)
TIME           CPU%     RSS(MiB)     VSZ(MiB)    THR
00:28:07      100.0         65.0        138.8      3
00:28:08      100.0         65.0        138.8      3
00:28:09      100.0         65.0        138.8      3
00:28:10      100.0         65.0        138.8      3
00:28:11      100.0         65.0        138.8      3
00:28:12      100.0         65.0        138.8      3
00:28:13      100.0         65.0        138.8      3
00:28:14      100.0         65.0        138.8      3
```

```
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py resize m --mem 256 --restart
stopped VM 'm' (pid 84501)
resized VM 'm': memory 64 -> 256 MiB, vCPUs 1 -> 1
restarted VM 'm' as pid 84890
```

```
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py stats m -n 20
VM 'm': pid 84890, 1 vCPU, 256 MiB allocated (100% CPU = one host logical CPU)
TIME           CPU%     RSS(MiB)     VSZ(MiB)    THR
00:28:30      100.0         81.0        330.8      3
00:28:31      101.0         97.0        330.8      3
00:28:32       99.0        113.0        330.8      3
00:28:33      100.0        113.0        330.8      3
00:28:34      101.0        129.0        330.8      3
00:28:35      100.0        129.0        330.8      3
00:28:36       99.0        145.0        330.8      3
00:28:37      100.0        145.0        330.8      3
00:28:38      101.0        161.0        330.8      3
00:28:39      100.0        161.0        330.8      3
00:28:40       99.0        177.0        330.8      3
00:28:41      100.0        177.0        330.8      3
00:28:42      101.0        193.0        330.8      3
00:28:43      100.0        193.0        330.8      3
00:28:44      100.0        209.0        330.8      3
00:28:45      100.0        209.0        330.8      3
00:28:46      100.0        225.0        330.8      3
00:28:47      100.0        225.0        330.8      3
00:28:48      100.0        241.0        330.8      3
00:28:49      100.0        241.0        330.8      3
```

```
● (base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py stats m -n 8
  VM 'm': pid 84890, 1 vCPU, 256 MiB allocated (100% CPU = one host logical CPU)
  TIME           CPU%     RSS(MiB)     VSZ(MiB)    THR
  00:30:36      100.0        257.0        330.8      3
  00:30:37      100.0        257.0        330.8      3
  00:30:38      100.0        257.0        330.8      3
  00:30:39      100.0        257.0        330.8      3
  00:30:40      100.0        257.0        330.8      3
  00:30:41      100.0        257.0        330.8      3
  00:30:42      100.0        257.0        330.8      3
  00:30:43      100.0        257.0        330.8      3
○ (base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$
main* 0↓ 4↑  ⊗ 0 ⚠ 0                                                    Ln 730, Col
```

آزمایش ششم

مقایسه VmSize و VmRSS

در این آزمایش دو ماشین با ۲۰۴۸ مگابایت RAM ساختیم.

ماشین اول برنامه cpu_burn.bin را اجرا می‌کرد و ماشین دوم برنامه mem_touch.bin را.

در ماشین اول، مقدار VmSize زیاد بود اما VmRSS بسیار کم باقی ماند.

در ماشین دوم، با لمس تدریجی صفحات حافظه، مقدار VmRSS افزایش پیدا کرد.

نتیجه: مقدار VmSize نشان‌دهنده اندازه فضای آدرس مجازی فرآیند است، اما VmRSS مقدار حافظه مقیم در RAM فیزیکی را نشان می‌دهد. بنابراین این دو مقدار الزاماً برابر نیستند.

```
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py run alloc cpu
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py stats alloc_cpu -n 5
VM 'alloc_cpu': pid 91610, 1 vCPU, 2048 MiB allocated (100% CPU = one host logical CPU)
TIME          CPU%     RSS(MiB)     VSZ(MiB)    THR
00:39:32      100.0          2.0       2122.8      3
00:39:33      100.0          2.0       2122.8      3
00:39:34      100.0          2.0       2122.8      3
00:39:35       99.9          2.0       2122.8      3
00:39:36       99.9          2.0       2122.8      3
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py list
NAME                         STATE          PID  VCPUS   MEM(MiB)  GUEST
alloc_cpu                    running      91610      1       2048  cpu_burn.bin
alloc_mem                    stopped          -      1       2048  mem_touch.bin
cpu                          stopped          -      4         64  cpu_burn.bin
cpu1                         stopped          -      4         64  cpu_burn.bin
cpu2                         stopped          -      2         64  cpu_burn.bin
cpu4                         stopped          -      4         64  cpu_burn.bin
hello                        halted           -      1         16  hello.bin
m                            stopped          -      1        256  mem_touch.bin
memory                       stopped          -      1        256  mem_touch.bin
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ grep -E 'VmSize|VmRSS' /proc/91610/status
VmSize:  2173708 kB
VmRSS:      2088 kB
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py stop alloc_cpu
stopped VM 'alloc_cpu' (pid 91610)
[minivmm] pid=91610 guest=/home/ahmad-shekib-haidari/Documents/CC/CC_HW1/minivmm-starter/guests/cpu_burn.bin mem=2048MiB vcpus=1
cpu_burn: all vCPUs are spinning (stop me with vmctl stop)
[minivmm] ===== VM exits seen by userspace (96.07 s) =====
reason                     vcpu0        total
IO                          4385         4385
INTR                           1            1
TOTAL                       4386         4386
[minivmm] VM stopped
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py run alloc_mem
started VM 'alloc_mem' as pid 92961 (1 vCPU, 2048 MiB) — it is a normal host process
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ python3 panel/vmctl.py stats alloc_mem -n 10
VM 'alloc_mem': pid 92961, 1 vCPU, 2048 MiB allocated (100% CPU = one host logical CPU)
TIME          CPU%     RSS(MiB)     VSZ(MiB)    THR
00:41:24      100.0        145.0       2122.8      3
00:41:25      100.0        145.0       2122.8      3
00:41:26      100.0        161.0       2122.8      3
00:41:27      100.0        161.0       2122.8      3
00:41:28      100.0        177.0       2122.8      3
00:41:29      100.0        193.0       2122.8      3
00:41:30       99.9        193.0       2122.8      3
00:41:31      100.0        209.0       2122.8      3
00:41:32       99.9        209.0       2122.8      3
00:41:33      101.0        225.0       2122.8      3
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$ grep -E 'VmSize|VmRSS' /proc/92961/status
VmSize:  2173708 kB
VmRSS:   1524772 kB
(base) ahmad-shekib-haidari@ahmad-shekib-haidari:~/Documents/CC/CC_HW1/minivmm-starter$
```

آزمایش هفتم

در این بخش سیستم‌عامل Alpine Linux را با QEMU/KVM اجرا کردیم.

ماشین مجازی دارای دو vCPU و ۵۱۲ مگابایت RAM بود.

سیستم‌عامل از دیسک مجازی alpine.qcow2 و بدون استفاده از فایل ISO بوت شد.

	specification
OS	alpine Linux 3.24.3
KERNEL	6.18.58-0-virt
ARC	x86_64
Vcpu amount	۲
RAM	۵۱۲ MiB
disk	alpine.qcow2

سیستم‌عامل با موفقیت اجرا شد و توانستیم وارد محیط آن شویم.

نتیجه: برخلاف minivmm ساده، QEMU امکانات لازم برای اجرای یک سیستم‌عامل کامل مانند Alpine Linux را فراهم می‌کند.

```
   OpenRC 0.63.2 is starting up Linux 6.18.58-0-virt (x86_64)

 * /proc is already mounted
 * Mounting /run ... [ ok ]
 * /run/openrc: creating directory
 * /run/lock: creating directory
 * /run/lock: correcting owner
 * Remounting devtmpfs on /dev ... [ ok ]
 * Mounting /dev/mqueue ... [ ok ]
 * Mounting security filesystem ... [ ok ]
 * Mounting debug filesystem ... [ ok ]
 * Mounting trace filesystem ... [ ok ]
 * Mounting persistent storage (pstore) filesystem ... [ ok ]
 * Mounting bpf filesystem ... [ ok ]
 * Starting busybox mdev ... [ ok ]
 * Scanning hardware for mdev ... [ ok ]
 * Loading hardware drivers ... [ ok ]
 * Loading modules ... [ ok ]
 * Setting system clock using the hardware clock [UTC] ... [ ok ]
 * Checking local filesystems  .../dev/vda3: clean, 2359/79040 files, 29404/316160 blocks
/dev/vda1: clean, 25/76912 files, 57981/307200 blocks
 [ ok ]
 * Remounting root filesystem read/write ... [ ok ]
 * Remounting filesystems ... [ ok ]
 * Activating swap devices ... [ ok ]
 * Mounting local filesystems ... [ ok ]
 * Configuring kernel parameters ... [ ok ]
 * Creating user login records ... [ ok ]
 * Setting hostname to ahmad-shekib-haidari from /etc/hostname ... [ ok ]
 * Starting networking ... *   lo ... [ ok ]
 *   eth0 ...udhcpc: started, v1.37.0
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.0.2.15, server 10.0.2.2
udhcpc: lease of 10.0.2.15 obtained from 10.0.2.2, lease time 86400
 [ ok ]
 * Seeding random number generator ... * Seeding 256 bits and crediting
 * Saving 256 bits of creditable seed for next boot
 [ ok ]
 * Starting busybox syslog ... [ ok ]
 * Starting busybox acpid ... [ ok ]
 * Starting busybox crond ... [ ok ]
 * Starting sshd ... [ ok ]

Welcome to Alpine Linux 3.24
Kernel 6.18.58-0-virt on x86_64 (/dev/ttyS0)

ahmad-shekib-haidari login: root
Password:
Welcome to Alpine!
```

.

```
ahmad-shekib-haidari login: root
Password:
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

ahmad-shekib-haidari:~# cat /etc/alpine-release
3.24.3
ahmad-shekib-haidari:~# uname -a
Linux ahmad-shekib-haidari 6.18.58-0-virt #1-Alpine SMP PREEMPT_DYNAMIC 2026-10-21 10:53:34 x86_64 Linux
ahmad-shekib-haidari:~# grep -i hypervisor /proc/cpuinfo | head -3
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm rep_good nopl xtopology cpuid tsc_known_fs
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm rep_good nopl xtopology cpuid tsc_known_fs
ahmad-shekib-haidari:~# dmesg | grep -i 'hypervisor detected'
```

.

آزمایش هشتم

برای بررسی QEMU با htop و vmctl stats

ابتدا فرآیند QEMU را پیدا کردیم و سپس با ابزارهای htop و vmctl stats مصرف منابع آن را بررسی کردیم.

برای مشغول کردن پردازنده مجازی، داخل Alpine دستور زیر را اجرا کردیم:

```
yes > /dev/null &
```

در htop یک نخ فعال مشاهده شد که تقریباً ۱۰۰ درصد CPU مصرف می‌کرد.

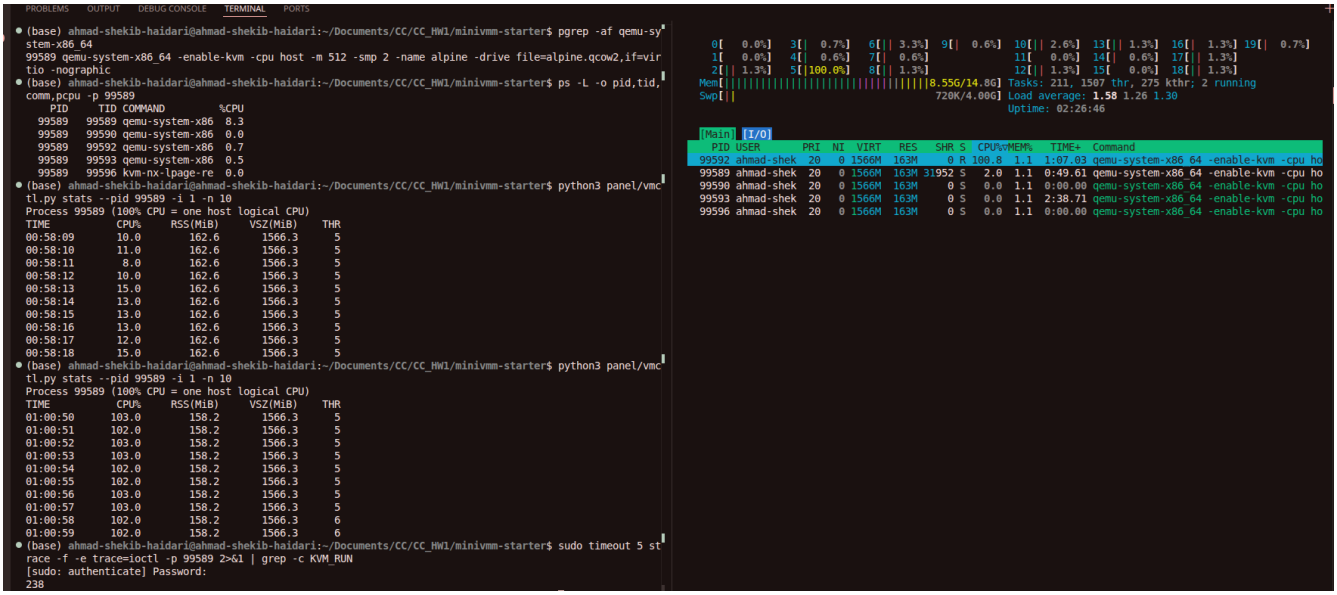
سپس با دستور زیر، فراخوانی‌های KVM_RUN را در بازه پنج‌ثانیه‌ای بررسی کردیم:

```
sudo timeout 5 strace -f -e trace=ioctl -p 99589 2>&1 | grep -c KVM_RUN
```

نتیجه: ۲۳۸ خط حاوی KVM_RUN

این عدد مربوط به خطوط مشاهده‌شده توسط strace است و لزوماً تعداد تمام VM Exitهای سخت‌افزاری نیست.

نتیجه‌گیری: QEMU نیز مانند minivmm از KVM استفاده می‌کند و می‌توان میزان مصرف CPU، حافظه و فعالیت نخ‌های آن را با ابزارهای لینوکس بررسی کرد.



.

جمع‌بندی و نتیجه‌گیری نهایی

در این پروژه با نحوه کار KVM و مجازی‌سازی در لینوکس آشنا شدیم. ابتدا یک هایپروایزر ساده را تکمیل کردیم و توانستیم برنامه‌های مهمان را اجرا کنیم و خروج‌های ماشین مجازی را بشماریم.

سپس با پنل vmctl ماشین‌های مجازی را مدیریت کردیم و اثر تغییر تعداد vCPU و حافظه را بررسی کردیم.

در پایان با QEMU/KVM سیستم‌عامل Alpine Linux را اجرا کردیم و مصرف منابع آن را مشاهده کردیم.

در مجموع این تمرین کمک کرد تفاوت فرآیند، نخ، پردازنده مجازی، حافظه مجازی و حافظه واقعی را به‌صورت عملی بهتر یاد بگیریم.

سوالات

سؤال ۱:

چون برنامه hello.bin برای چاپ متن از دستور OUT استفاده می‌کند و هر بار اجرای این دستور باعث خروج I/O می‌شود. در آزمایش ما ۵۴ خروج IO و یک خروج HLT ثبت شد؛ یعنی مجموعاً ۵۵ خروج. تعداد دقیق خروج‌های IO به تعداد دستورهای ورودی و خروجی اجراشده در برنامه مهمان بستگی دارد.

سؤال ۲:

ابتدا مهمان دستور OUT را اجرا می‌کند و VM Exit اتفاق می‌افتد. سپس KVM اطلاعات خروج را در ساختار kvm_run قرار می‌دهد. برنامه minivmm با تابع handle_io داده را دریافت می‌کند و اگر پورت سریال باشد، آن را روی ترمینال چاپ می‌کند. در پایان با اجرای دوباره KVM_RUN، مهمان از دستور بعدی ادامه می‌دهد.

سؤال ۳:

کد مهمان در حالت VMX non-root و سطح دسترسی ring 0 اجرا می‌شود. برنامه minivmm در حالت VMX root و ring 3 قرار دارد و بخش KVM در هسته لینوکس با VMX root و ring 0 کار می‌کند.

پردازنده و KVM دسترسی مهمان به منابع میزبان را محدود می‌کنند. بنابراین مهمان دسترسی مستقیم و نامحدود به حافظه و دستگاه‌های میزبان ندارد؛ البته این جداسازی به معنی امنیت صددرصد در برابر آسیب‌پذیری‌ها نیست.

سؤال ۴:

با دستور kill -STOP فرآیند minivmm متوقف می‌شود و نخ‌های vCPU آن دیگر اجرا نمی‌شوند؛ بنابراین مصرف CPU کاهش می‌یابد. با دستور kill -CONT فرآیند دوباره ادامه پیدا می‌کند و مصرف CPU افزایش می‌یابد.

این آزمایش نشان می‌دهد زمان اجرای vCPUها توسط زمان‌بند لینوکس میزبان کنترل می‌شود.

سؤال

چون VmSize اندازه فضای آدرس مجازی فرآیند را نشان می‌دهد، اما VmRSS فقط حافظه‌ای را نشان می‌دهد که واقعاً در RAM مقیم است.

در آزمایش ما، ماشین cpu_burn با ۲۰۴۸ مگابایت RAM، مقدار VSZ حدود ۲۱۲۲٫۸ مگابایت داشت، ولی RSS فقط ۲ مگابایت بود.

این ویژگی به سرویس‌دهنده ابری اجازه می‌دهد با درنظرگرفتن میزان مصرف واقعی و سیاست‌های حافظه، منابع را بهتر مدیریت کند و در بعضی شرایط از overcommit استفاده کند؛ البته مصرف زیاد هم‌زمان ماشین‌ها ممکن است باعث کمبود حافظه شود.

سؤال ۶:

بله. چون vmctl.py فقط نقش پنل مدیریت را دارد و ماشین‌های مجازی را به‌صورت فرآیندهای مستقل minivmm اجرا می‌کند.

بعد از اجرای ماشین، بسته شدن پنل باعث توقف خودکار فرآیند ماشین مجازی نمی‌شود. در پروژه ما اطلاعات ماشین‌ها نیز در فایل JSON ذخیره می‌شوند.

سؤال ۷:

هر دو از KVM لینوکس برای اجرای پردازنده‌های مجازی با کمک سخت‌افزار استفاده می‌کنند و می‌توانند خروج‌های VM را مدیریت کنند.

اما QEMU امکانات بیشتری مانند شبیه‌سازی دیسک، کارت شبکه، دستگاه‌های سخت‌افزاری و محیط لازم برای بوت سیستم‌عامل را فراهم می‌کند.

به همین دلیل در آزمایش ما با QEMU توانستیم سیستم‌عامل کامل Alpine Linux را نصب و اجرا کنیم، ولی minivmm فقط برای اجرای برنامه‌های ساده مهمان مانند hello.bin طراحی شده است.